

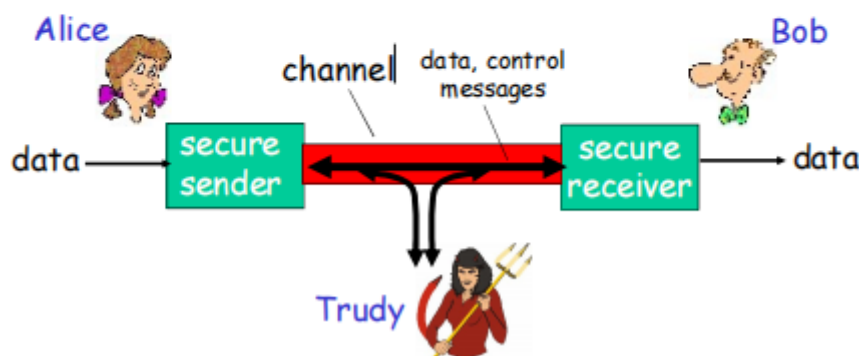
第八章 网络安全

1.什么是网络安全

- 机密性：只有发送方和预订的接收方能否理解传输的报文内容
 - 发送方加密报文
 - 接收方解密报文
- 认证：发送方和接收方需要确认对方的身份
- 报文完整性：发送方、接收方需要确认报文在传输过程中或者事后没有被改变
- 访问控制和服务的可用性：服务可以接入以及对用户而言是可用的

朋友和敌人: Alice, Bob, Trudy

- 网络安全世界比较著名的模型
- Bob, Alice (lovers!) 需要安全的通信
- Trudy (intruder) 可以截获，删除和增加报文



网络中的坏蛋

Q: "bad guy"可以干什么?

A: 很多!

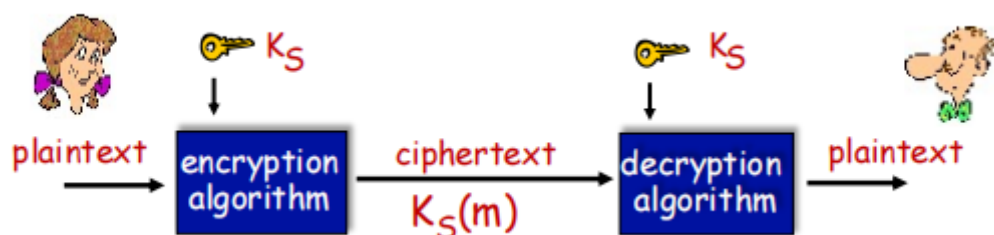
- **窃听:** 截获报文
- **插入:** 在连接上插入报文
- **伪装:** 可以在分组的源地址写上伪装的地址
- **劫持:** 将发送方或者接收方踢出，接管连接
- **拒绝服务:** 阻止服务被其他正常用户使用 (e.g., 通过对资源的过载使用)

2.加密原理

对称密钥密码学：发送方和接收方的密钥相同

公开密钥密码学：发送方使用接收方的公钥进行加密，接收方使用自己的私钥进行解密

2.1 对称密钥加密



2.1.1 替换密码

单码替换：将一个字母替换成另一个字母

多码替换：

- n substitution ciphers, $M_1, M_2, \dots, M_n$
- cycling pattern:
 - e.g., $n=4$: M_1, M_3, M_4, M_3, M_2 ; M_1, M_3, M_4, M_3, M_2 ; ..
- for each new plaintext symbol, use subsequent substitution pattern in cyclic pattern
 - dog: d from M_1 , o from M_3 , g from M_4

- 🔑 **Encryption key:** n substitution ciphers, and cyclic pattern
 - key need not be just n -bit pattern

2.1.2 DES

Data Encryption Standard

56-bit 对称密钥, 64-bit明文输入

更安全的DES：使用3个Key, 3重DES

2.1.3 AES

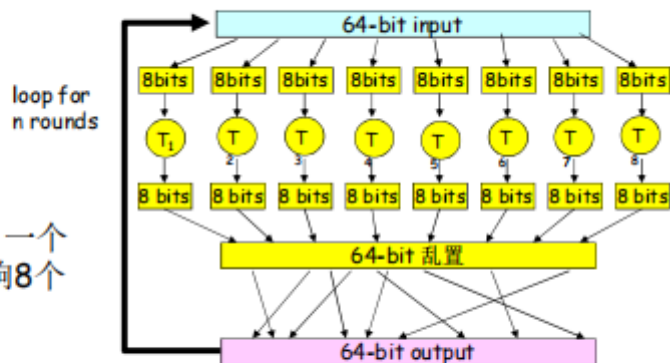
Advanced Encryption Standard

数据128bit成组加密

2.1.4 块密码

块密码

- 一个循环：一个输入bit影响8个输出bit

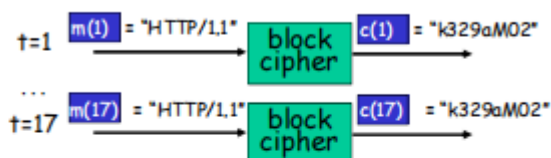


- 多重循环：每个输入比特影响所有的输出bit
- 块密码：DES, 3DES, AES

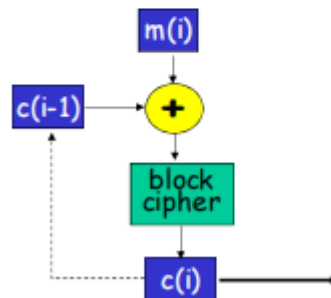
2.1.5 密码块链

密码块链

- 密码块：如果输入块重复，将会得到相同的密文块



- 密码块链：异或第 i 轮输入 $m(i)$ ，与前一轮的密文， $c(i-1)$
 - $c(0)$ 明文传输到接收端
 - what happens in "HTTP/1.1" scenario from above?

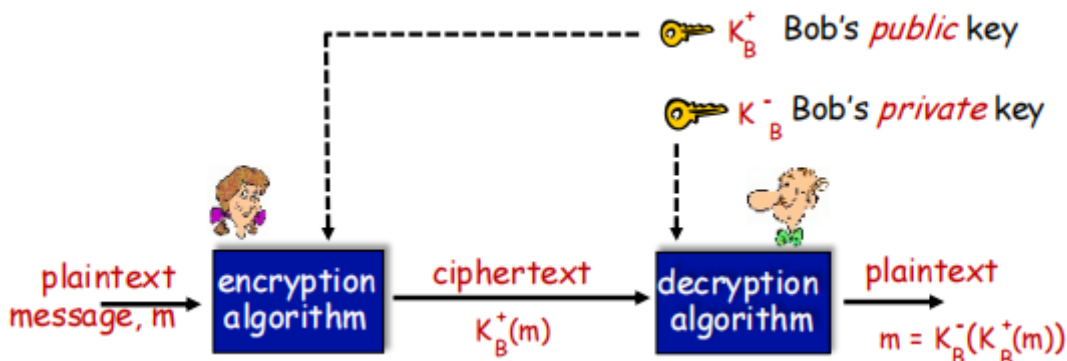


2.2 公开密钥加密

发送方和接收方无需共享密钥

一个实体的公钥公之于众

私钥只有它自己知道



公开密钥加密算法要求:

1. 需要 $K_B^+(\cdot)$ 和 $K_B^-(\cdot)$, 满足 $K_B^-(K_B^+(m)) = m$
2. 给定一个公钥 K_B^+ 推出私钥 K_B^- 计算上不可行

2.2.1 RSA选择密钥

Rivest, Shamir, Adelson Algorithm

1. 选择2个很大的质数 p, q . e. g., 1024bits
2. 计算 $n = pq, z = (p-1)(q-1)$
3. 选择一个 e (要求 $e < n$)和 z 没有一个公共因子, 互素
4. 选择 d 使得 $ed - 1$ 正好能够被 z 整除
5. 公钥 (n, e) . 私钥 (n, d) .

2.2.2 RSA加密解密

1. 加密一个bit模式, m , 如此计算:

$$c = m^e \mod n$$

2. 对接收到的密文 c 解密, 如此计算:

$$m = c^d \mod n$$

$$m = (m^e \mod n)^d \mod n!$$

一个数论定理: 如果 p, q 都是素数, $n=pq$,那么:

$$x^y \mod n = x^{y \mod (p-1)(q-1)} \mod n$$

$$(m^e \mod n)^d \mod n = m^{ed} \mod n = m^{ed \mod (p-1)(q-1)} \mod n = m^1 \mod n = m$$

2.2.3 RSA的另外一个特性

$$K_B^-(K_B^+(m)) = m = K_B^+(K_B^-(m))$$

2.2.4 为什么RSA是安全的?

因为对一个大数寻找因数是非常困难的。

2.2.5 RSA实践: 会话密钥

DES比RSA速度快多了。

发送方和接收方首先通过RSA交换对称会话密钥 K_S ,建立安全连接, 然后两者再使用 K_S 进行会话。

3.认证

5种认证的情形, 都可能被攻击。

4.报文完整性

4.1 数字签名

发送方数字签署了文件，前提是他是文件的拥有者/创建者

特性：可验证性，不可伪造性，不可抵赖性

简单的对m的数字签名：Bob使用他自己的私钥对m进行签署，创建数字签名 $K_B^-(m)$

Alice收到报文m以及数字签名 $K_B^-(m)$ 。Alice使用Bob的公钥 $K_B^+(m)$ 对 $K_B^-(m)$ 验证。

4.2 报文摘要

使用报文摘要的原因：对长报文进行公开密钥加密算法需要消耗大量时间。

对m使用哈希函数H，获得固定长度的报文摘要 $H(m)$ 。

哈希函数的特性：

- 多对1
- 结果固定长度
- 给定一个报文摘要x，反向计算出原报文在计算上是不可行的（即可以通过哈希函数求摘要，但是通过摘要反向求原报文不可行）

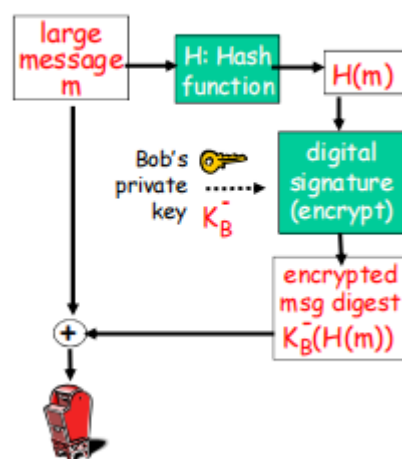
4.3 Internet校验和：弱的哈希函数

特性：

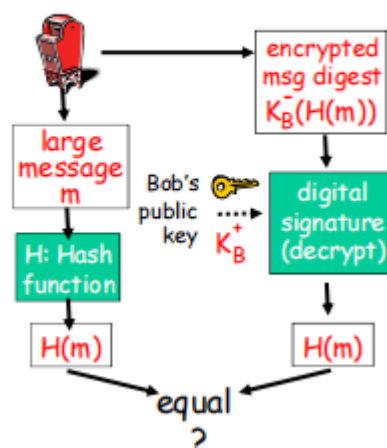
- 产生报文m的固定长度的摘要
- 多对1的
- 但是不同的报文可能对应相同的校验和

4.4 数字签名=对报文摘要进行数字签署

Bob 发送数字签名的报文：



Alice 校验签名和报文完整性：



4.5 哈希函数算法

- MD5
- SHA-1

5. 密钥分发和证书

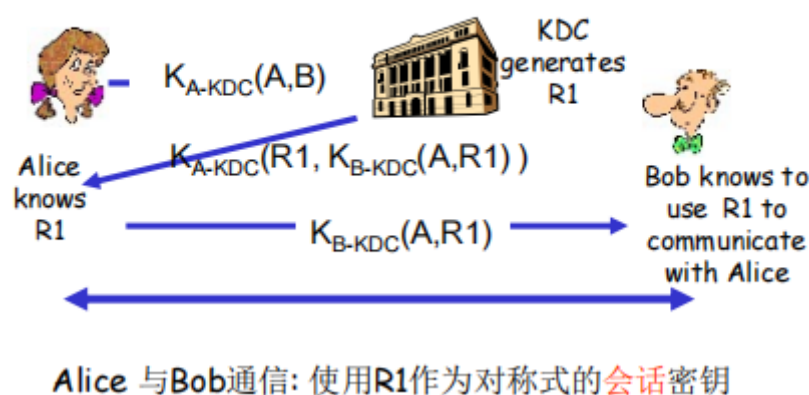
5.1 可信赖中介

- 对称密钥问题: KDC (Key Distribution Center)
- 公共密钥问题: CA (Certification Authority)

5.2 KDC

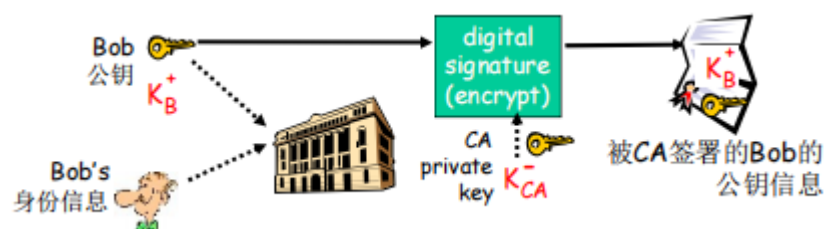
KDC: 服务器和每一个注册的用户之间都共享一个对称式的密钥

Q: KDC如何使得 Bob和Alice在和对方通信前, 就对称式会话密钥达成一致?

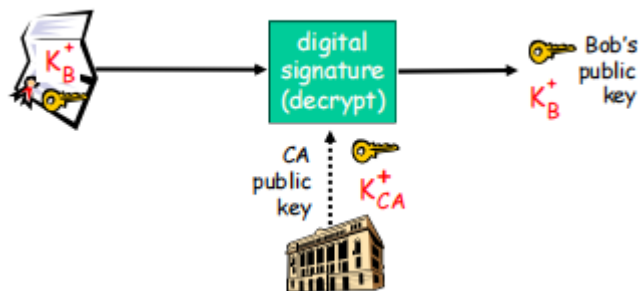


5.3 CA

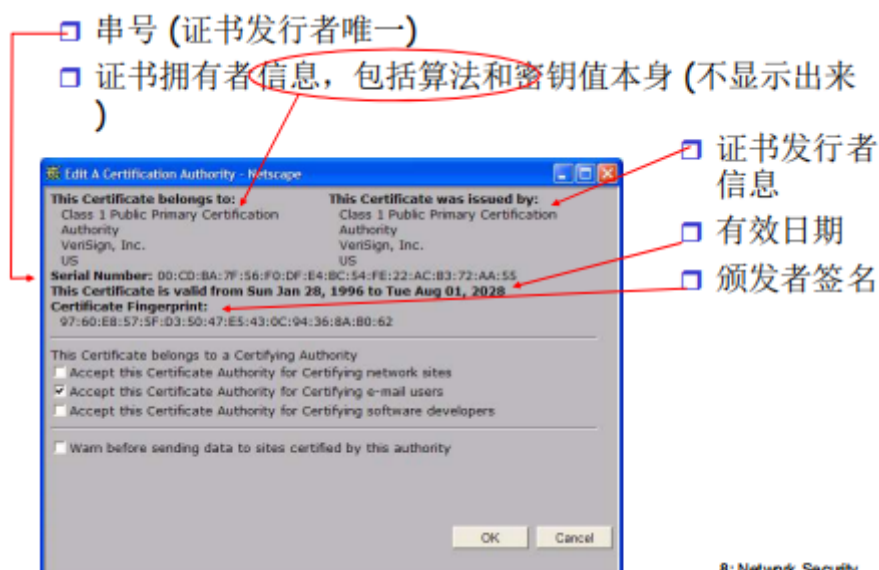
- CA: 将每一个注册实体E和他的公钥捆绑
- E (人或者路由器) 到CA那里注册他的公钥
 - E提供给CA自己身份的证据
 - CA创建一个证书, 捆绑了实体信息和他的公钥
 - 证书包括了E的公钥, 而且是被CA签署的 (被CA用自己的私钥加密的)



- 当Alice需要拿到Bob的公钥
 - 获得Bob的证书
 - 对Bob的证书使用CA的公钥来验证



- 证书包括：



8: Network Security 53

5.4 信任树

- 根证书：是未被签名的公钥证书或自签名的证书
 - 拿到一些CA的公钥
 - 渠道：安装OS自带的数字证书；从网上下载
- 信任树：
 - 信任根证书CA颁发的证书，拿到了根CA的公钥
 - 由根CA签署的给一些机构的数字证书，包含了这些机构的数字证书
 - 由于你信任了根，从而能够可靠地拿到根CA签发的证书，可靠地拿到这些机构的公钥

6.各个层次的安全性

6.1 安全电子邮件

6.2 使TCP连接安全：TLS

6.2.1 简介

- 广泛部署的安全协议
 - 被几乎所有浏览器和Web服务器采用，如https
 - 被QUIC采用（协作），握手时的认证和密钥集协商，之后的加密数据传输

- https
- 能够提供：
 - 机密性：采用对称加密
 - 完整性：通过加密的散列值
 - 认证：通过公开密钥加密体系
- 历史：
 - 早期的研究和实现：安全网络编程，安全套接字接口
 - SSL3.0->TLS 1.0
 - TLS 1.2
 - TLS 1.3

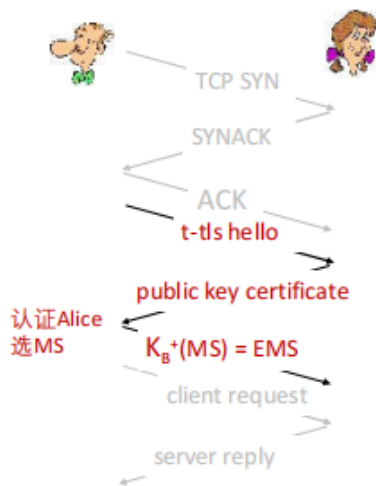
6.2.2 t-tls

安全通信的步骤：

- 握手：Alice和Bob使用各自证书，私钥来认证对方，交换或者创建共享密钥（对称密钥）
- 密钥导出：Alice和Bob采用共享密钥来生成一个密钥集
- 数据传输：要传输的数据被分成若干记录的序列（数据块）
- 连接关闭：采用特殊的报文安全关闭连接

1. 初始化握手：

t-tls:初始化握手



t-tls 握手阶段：

- Bob 建立和Alice的TCP连接
- Bob 验证的Alice的身份
- Bob发送给Alice 主密钥 key_A^+ (MS), 用来生成TLS会话的所有密钥
- 可能存在的问题：
 - 在通信之前需要花费3 RTT（包括TCP连接建立）

2. 密钥生成：

t-tls:密钥生成

- 在多个加密操作中采用同一个密钥：不安全
 - 采用不同的密钥用于：报文认证(MAC)和加密
- 4 keys:
 - K_c = 客户端到服务器的加密密钥
 - M_c = 客户端到服务器的MAC密钥
 - K_s = 服务器到客户端的加密密钥
 - M_s = 服务器到客户端的MAC密钥
- 密钥由密钥key derivation function (KDF)导出函数导出
 - KDF创建密钥的输入：主密钥MS+（可能的）一些附加随机数据
 - 破除主密钥MS 和 4Keys的固定对应关系

Security: 8-6

3. 加密数据：

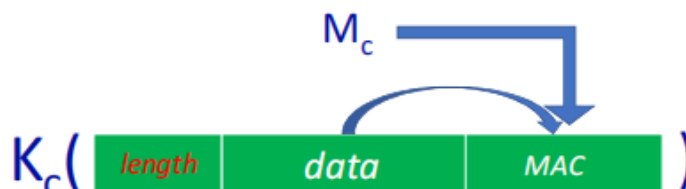
t-tls:加密数据

- 回顾：TCP提供数据字节流传输的服务
- Q: t-tls可以对数据进行字节流流式加密？就像TCP Socket
 - A: 如是流式加密，完整性校验的MAC只能在最后，那么只有所有数据都被传输完毕才能够进行完整性校验！
 - 如：对于即时通信（持续时间很长），在终端显示这些字符之前，如何对 所有字节 进行完整性校验？
 - 方案:将字节流分割成一个个的记录
 - 每个记录C-S携带该记录的MAC，hash（采用 M_c ）
 - 接收端可以对每个到达的记录进行完整性校验
- t-tls记录采用对称密钥 K_c 加密，交付给TCP传输： $MAC=HASH(data+ M_c)$



Security: 8-7

- Q: 在记录中，接收端要区分数据与MAC
 - 采用可变长记录，用length指示数据长度
 - $MAC=MAC(data, M_c)$; Data和MAC加密传输
 - M_c 加入MAC的运算，MAC不仅仅依赖于data，更安全
- t-tls记录采用对称密钥加密，用密钥 K_c ，交付给TCP传输：



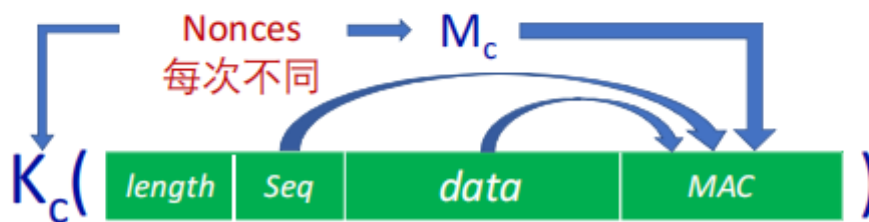
Security: 8-9

- 针对数据流的可能攻击？
 - **重排攻击**：中间攻击者可以捕获、重新排序攻击
 - Client→Server 2个TCP段
 - 中间人Trudy颠倒2个Tcp段的序号（TCP段没有加密，可以修改TCP头部序号）
 - Server TCP收到，交TLS，上交应用
 - 应用无法检查出颠倒的问题
- 方案：将记录的**序号**放到MAC中， $MAC = MAC(M_c, sequence || data)$
 - 发送者TLS使用记录序号0, 1, 2...（每发送一个+1）
 - 记录序号加入到MAC的运算



Security: 8- 10

- 针对数据流的可能攻击？
 - **重放攻击**：攻击者重放所有的记录
- 方案：采用不重数**nonce**
 - 每次TLS握手，采用不同的nonces
 - 密钥4keys依赖于nonce，每次连接密钥不同
 - 重放攻击不起作用



4. 连接关闭：

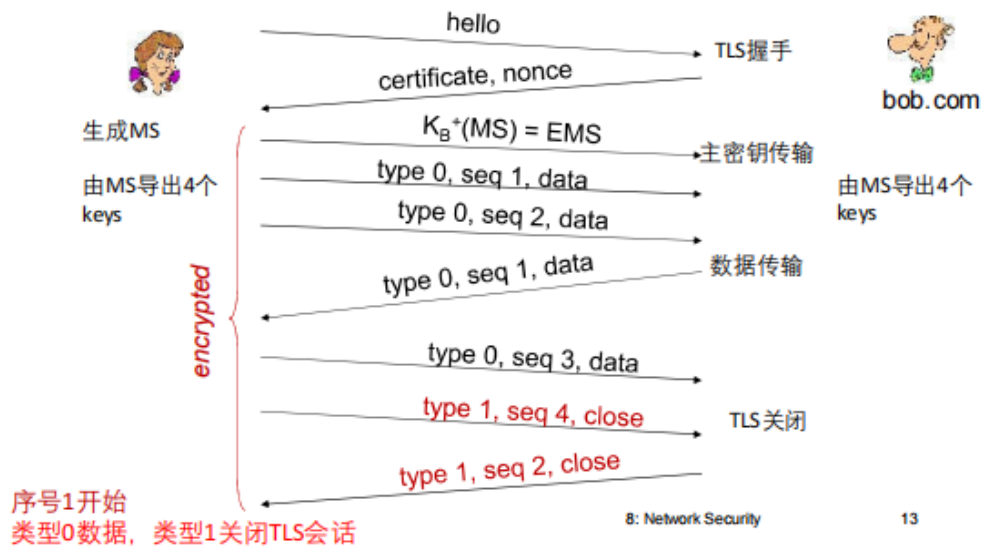
t-tls: 连接关闭

- 截断攻击：
 - 攻击者伪造TCP连接的报文段；伪造TCP连接关闭
 - 连接过早地被关闭，一方或者双方只接收到了比实际少的数据
- 方案：*TLS*记录类型，一个类型用于关闭
 - TLS类型0**数据**；类型1**用于关闭**
 - TLS会话记录type=1传输之后，才是TCP连接关闭
 - 本质上：上层TLS协议中包括下层TCP协议的一些控制信息
- MAC采用数据，类型type，和序号 来计算
- $MAC = MAC(M_c, sequence || type || data)$



Security: 8- 12

t-tls 过程：



6.2.3 TLS加密套件

TLS加密套件：密钥生成算法，加密算法，MAC算法，数字签名算法

TLS1.3提供了比TLS1.2支持更少的加密套件（防止降级攻击）。

- Diffie-Hellman (DH) 算法进行密钥交换，而不是DH或RSA
- 组合加密和认证算法，而不是加密之后再认证
- HMAC采用SHA加密散列函数

TLS 加密套件

密码套件的名称和其安全性属性的示例

密码套件名称	身份验证	密钥交换	密码	MAC	PRF
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256	RSA	ECDSA	AES-128-GCM	—	SHA256
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384	ECDSA	ECDSA	AES-256-GCM	—	SHA384
TLS_DHE_RSA_WITH_3DES_EDE_CBC_SHA	RSA	DHE	3DES-EDE-CBC	SHA1	协议
TLS_RSA_WITH_AES_128_CBC_SHA	RSA	RSA	AES-128-CBC	SHA1	协议
TLS_ECDHE_ECDSA_WITH_AES_128_CCM	ECDSA	ECDSA	AES-128-CCM	—	SHA256



https://blog.csdn.net/m0_37621078/article/details/106028622

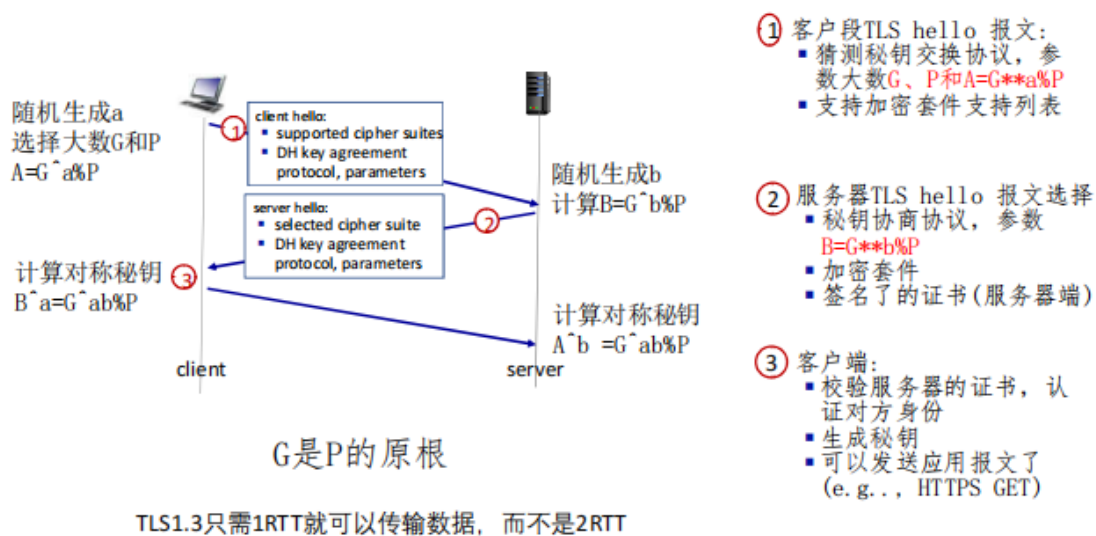
Security: 8- 17

6.2.4 真实TLS：握手

目的：

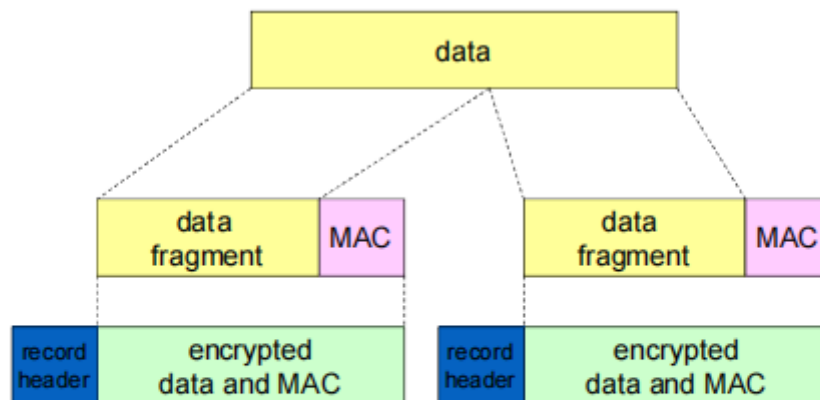
- 服务器认证（客户端认证服务器）
- 协商：就加密算法和MAC算法协商一致
- 建立密钥
- 客户端认证

TLS 1.3 握手：1RTT



Security: 8- 21

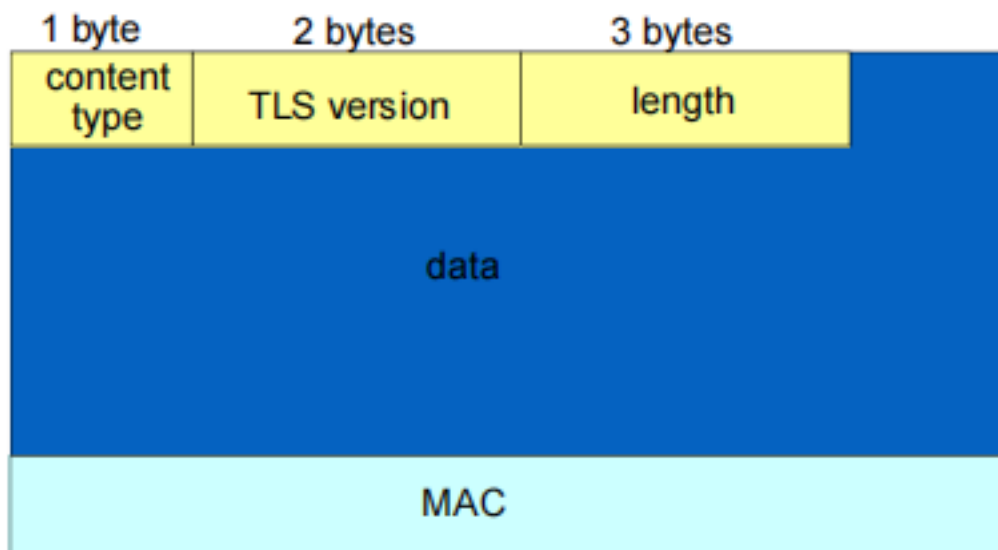
6.2.5 TLS 记录格式



record header: content type; version; length

MAC: includes sequence number, MAC key M_x

fragment: each TLS fragment 2^{14} bytes (~16 Kbytes)



数据和MAC都是被加密的(对称算法)

6.3 IPsec

6.3.1 位置

- 在两个网络实体（主机或者路由器）之间提供安全性
- IP之上、传输层协议之下
- 发送实体加密数据报载荷，载荷可以是：
TCP段，UDP数据报，ICMP报文，OSPF报文
- 所有从发送端实体到其他接收方的数据都可以被隐藏
- 地毯覆盖：IP的载荷是经过加密，认证，完整性检查的

6.3.2 特性和组成

- 安全性：机密性、完整性、可认证性、重放攻击保护
- 2个IPSec子协议：
 - Authentication Header(AH)协议：提供源端的可认证性和数据完整性，但是不提供机密性
 - Encapsulation Security Protocol (ESP)：提供源端可认证性，数据完整性和机密性；比AH用的广泛

6.3.3 模式

- 传输模式：主机之间；只有数据报载荷被加密和认证
- 隧道模式：路由器之间；整个数据报都被加密和认证；加密的数据报被封装在一个新的数据报中，新的IP头部；2个路由器之间就像有一个隧道一样间2个网络连接起来

6.3.4 SAs(Security Associations)

安全关联

- 在发送数据前，要在发送实体和接收实体之间建立安全关联
 - SA复杂：一个方向一个安全关联
 - 单向安全数据通信的关系
- 发送端、接收端实体维护SA的状态信息
 - 维护通信的状态
 - IP是无连接的：IPSec是面向连接的
- VPN一共有多少个SA，一个总部一个分支和n个旅行中的销售： $2+2n$

安全关联 (SAs)

R1为SA存储：

- 32-bit identifier: *Security Parameter Index (SPI)*
- 源SA接口 (200.168.1.100)
- 目标 SA 接口 (193.68.2.23)
- 采用的加密类型(e. g. , 3DES with CBC)
- 加密的密钥
- 完整性校验类型(e. g. , HMAC with MD5)
- 认证的key

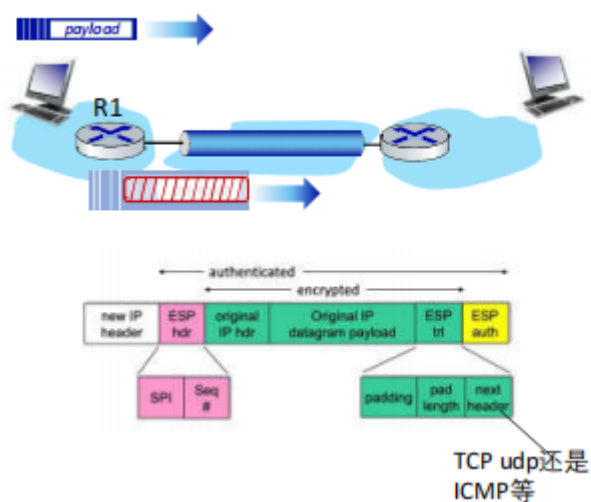


6.3.5 SAD

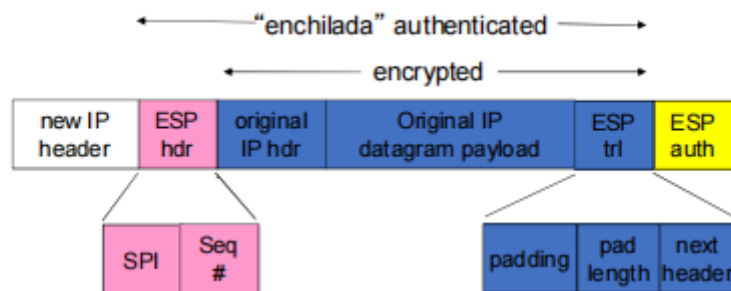
- 端节点将SA的状态维护在SAD中，可以使用它们在处理过程中访问到所用的信息
- 当发送IPSec数据报时，R1访问SAD来决定如何处理数据报
- 当IPSec数据报到达R2时，R2检查IPSec数据报的SPI，用该值检索SAD，对应地处理该数据报

6.3.6 ESP隧道模式：R1动作

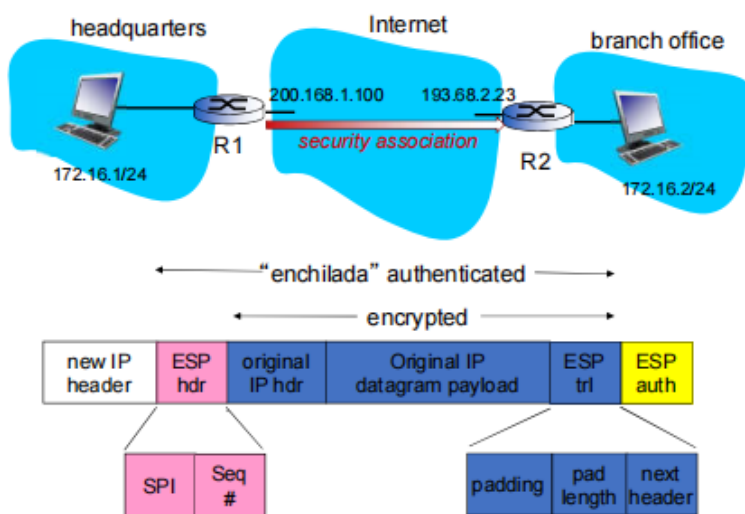
ESP、隧道模式



IP数据报封装在IPSec中：



- ESP 尾部：块加密需要填充成块的整数倍
- ESP 头部：
 - SPI，让接收实体知道如何处理
 - 序号，防止重放攻击
- ESP中的MAC认证字段，采用共享密钥创建



1. 在原始数据报（包括原始数据报的头部字段）的尾部附加“ESP尾部”字段
2. 采用SA指定的算法和key加密以上结果
3. 将被加密部分的前面，插入ESP头部，创建“enchilada”
4. 对全部的“enchilada”创建鉴别MAC，采用SA中指定的算法和key，在“enchilada”后面附上MAC，形成载荷
5. 创建新的IP头部，包括典型IPv4的头部字段，插在载荷的前面
 - 协议字段：50，标识 ESP来处理
 - 源IP R1，目标IP R2

6.3.7 IPsec序号

- 对于一个新的SA，发送端初始化该序号为0
- 每一次在SA上发送数据报
 - 发送方增加序号的值
 - 将该序号值放入seq字段中
- 目标：
 - 阻止嗅探和重放攻击
 - 如果接收到重复的，经过认证的IP数据报可以中断服务
 - 攻击者只改序号是不行的，经过认证的，对应的MAC字段也需要修改
- 方法：
 - 目标端检查重复
 - 不维护所有接收到分组的状态，而是采用窗口范围内的检查

6.3.8 SPD

安全策略数据库：

- 策略：给定一个数据报，发送实体知道它是否该使用IPSec，还是普通IP数据报
- 需要知道采用哪个SA关联的数据
 - 可能会用到：源和目标的IP地址，协议号
- 在SPD中的信息指示：对于到来的数据报到底该做什么
- SAD中的信息指示：如何做
 - 按照具体参数对进出的数据报进行IPSec的处理

6.3.9 IPSec安全性分析

IPSec服务安全性分析



- 假设Trudy在R1和R2之间，她不知道keys.
 - Trudy能够看到数据报的原始内容吗？源IP、目标IP地址，传输层协议，应用端口号能够探知吗？
 - 原始数据报加密，IP地址，端口号信息不可探知
 - 可以更改其中的某些bits，而且不被检测到吗？
 - 序号在IPSec分组中，有完整性检查机制
 - 伪装成R1，采用R1的IP地址呢？
 - 也通不过完整性检查，没办法形成正确的MAC
 - 重放一个数据报呢？
 - 有序号，通不过完整性检查

6.3.10 IKE: Internet Key Exchange

IKE:自动建立SA

- 交换证书（认证），协商，鉴别和加密算法
- 交换用于生成Key的信息，生成keys
- 建立SA

IKE过程：

IKE：阶段1

- 条件：如果双方没有建立起VPN的**管理连接IKE SA**
- 目的：建立管理连接IKE SA(双向，不同于SA)
 - 安全参数**协商**（加密算法，MAC算法）
 - **密钥生成**（HF算法），双方共享IKE SA的密钥
 - 相互**认证对方**（对称方式，RSA非对称方式）
- 模式：主模式和侵略模式（贪婪模式）
 - 主模式提供身份保护，更加弹性化
 - 贪婪模式采用更少的报文，更快

IKE：阶段2

- 目的：在阶段1建立的IKE SA基础上，安全地建立2个**数据连接SA**(2个单向)
- 结果：
 - 协商加密算法，鉴别算法
 - 建立加密密钥，鉴别密钥
 - 周期性的对数据连接进行密钥更新

IKE：PSK和PKI

- 相互认证对方身份（证明你是谁）
 - pre-shared secret (PSK) ，对称加密体系
 - public/private keys and certificates (PKI) ，公开密钥加密体系
- PSK：双方都从一个secret开始
 - 运行IKE来认证对方，而且建立起 IPsec SAs（每个方向一个），包括加密和认证的keys
- PKI：双方开始于public/private key 对和证书
 - 运行IKE来相互认证对方，获得 IPsec SAs（每个方向一个）
 - 和SSL握手类似

IKE交换报文用于：协商算法，生成和交换keys，生成SPI，自动建立SA

关于IPSec更详细的介绍参见:[IPSec协议](#)

6.4 无线和移动网络中的安全

6.4.1 802.11 WLAN

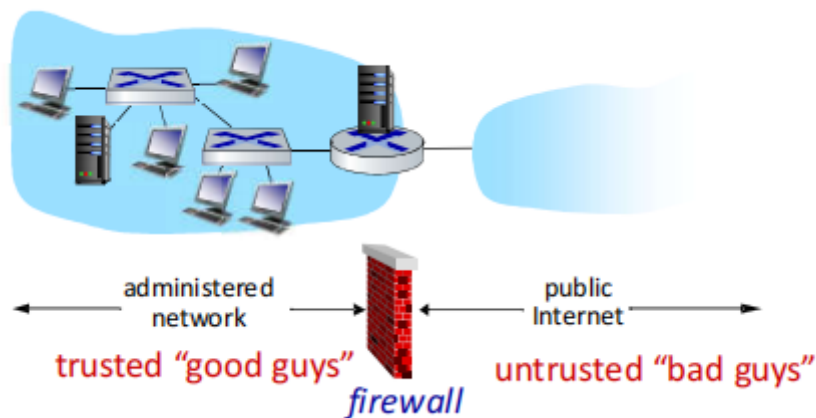
6.4.2 4G/5G

7.防火墙和IDS

7.1 防火墙

7.1.1 定义

将组织内部网络和互联网隔离开来,按照规则允许某些分组通过,或者阻塞掉某些分组



7.1.2 必要性和分类

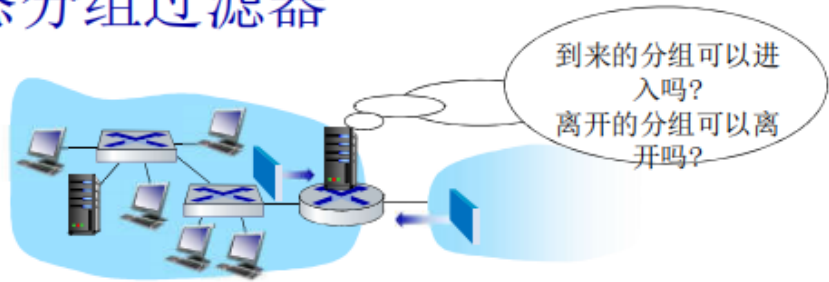
必要性:

- 组织拒绝服务攻击
- 组织非法的修改/对非授权内容的访问
- 只允许认证的用户能访问内部网络资源

类型:

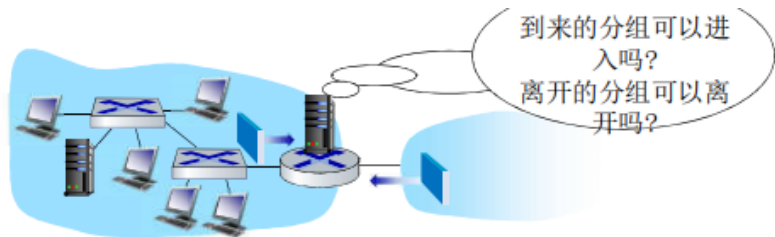
- 无状态分组过滤器
- 有状态分组过滤器
- 应用网关

无状态分组过滤器



- 内部网络通过配置防火墙的路由器连接到互联网上
- 路由器对分组逐个过滤，根据分组相应字段匹配到规则相应自字段来决定转发还是丢弃
 - 源IP地址，目标IP地址
 - TCP/UDP源和目标端口
 - ICMP报文类别
 - TCP SYN 和ACK bits

示例:



- 例1: 阻塞进出的数据报: 只要拥有IP协议字段 = 17, 而且源/目标端口号 = 23.
 - 结果: 所有的进出UDP流 以及TCP 上telnet连接分组都被阻塞掉
- 例2: 阻塞进入内网的TCP段: 它的ACK=0.
 - 结果: 阻止外部客户端主动和内部网络的主机建立TCP连接, 但允许内部网络的客户端主动和外部服务器建立TCP连接

策略	防火墙设置
不允许外部的web进行访问	阻塞掉所有外出具有目标端口80的IP分组
不允许来自外面的TCP连接, 除非是机构公共WEB服务器的连接	阻塞掉所有进来的TCP SYN分组, 除非 130.207.244.203, port 80
阻止Web无线电占用可用带宽.	阻塞所有进来的UDP分组 - 除非 DNS 和路由器广播
阻止你的网络被smurf DoS所利用	阻塞掉所有到达广播地址 (130.207.255.255) 的ICMP分组.
阻止内部网络被tracerout, 从而得到你的网络拓扑	阻塞掉所有外出的 ICMP TTL过期的流量

Access Control Lists

ACL:规则的表格,自动向下和输入的分组进行匹配(action,condition)对,有点像OenFlow转发表

action	source address	dest address	protocol	source port	dest port	flag bit
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	---
deny	all	all	all	all	all	all

7.1.4 有状态分组过滤器

■ 无状态分组过滤器：重型工具？

- 防火墙会让“无意义”的分组通过，例如：dest port = 80, ACK bit set
- 该TCP连接甚至都没建立起来：

action	source address	dest address	protocol	source port	dest port	flag bit
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK

■ 有状态的分组过滤器：跟踪每个TCP连接的状态

- 跟踪TCP连接建立（SYN），拆除（FIN）：然后才让相应后续分组通过
- 防火墙上的非活跃连接会超时，不再允许相应的分组通过防火墙

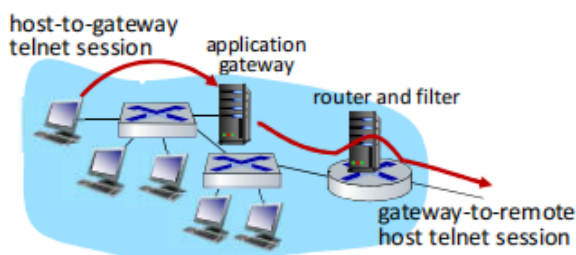
相应的ACL:

ACL增强：在允许分组通过之前需要检查连接状态表

action	source address	dest address	proto	source port	dest port	flag bit	check connection
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any	
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK	X
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---	
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	---	X
deny	all	all	all	all	all	all	

7.1.5 应用程序网关

- 根据应用数据的内容来过滤进出的数据报，就像防火墙根据IP/TCP/UDP字段来过滤一样
- 例子：允许特定的内部站点登录到外部服务器，但不是直接登录



1. 需要所有的telnet用户通过网关来telnet
2. 对于认证的用户而言，网关建立和目标主机的telnet connection，网关在2个连接上进行中继
3. 路由器过滤器将所有不是来自网关的telnet分组全部过滤掉

7.1.6 局限性

- **IP spoofing:** 路由器不知道数据报是否真的来自于分组源地址声称的IP
- 应用网关：如果有多个应用需要控制，就需要有多个应用程序网关
- 而且客户端软件需要知道如何连接到这个应用网关
 - e.g., 必须在Web browser中配置网络代理的Ip地址
- 过滤器对UDP段所在的分组，或者全过或者全都不过
- 折中：外部通信的便利性vs安全的级别
- 很多高度保护的站点仍然受到攻击的困扰

7.2 IDS:入侵检测系统

- 分组过滤：
 - 对TCP/IP头部字段进行检查
 - 不检查会话分组间的相关性
- IDS：
 - 深入分组检查:检查分组的内容
 - 检查分组间的相关性,判断是否是有害的分组
 - 端口扫描
 - 网络映射
 - DoS攻击

- 多个IDSs: 不同的网段, 放置探针, 根据需要进行不同类型的检查

